

MailSense: Actionable Email Detection Using NLP

Formal Academic Project Report

Md Ariful Alam Mahim (Roll: 2107023)

Md Jubair Husain (Roll: 2107029)

September 2026

1. Introduction

Electronic mail can contain requests, deadlines, confirmations, notices, and other content that may require a recipient's attention. MailSense addresses the focused task of identifying whether a piece of email content is actionable. The project implements and compares three binary text-classification pipelines: a traditional TF-IDF plus Logistic Regression baseline, a pretrained Word2Vec plus bidirectional LSTM (BiLSTM) neural model, and a fine-tuned pretrained BERT model (bert-base-uncased).

The repository describes the input records as individual email sentences or snippets rather than complete messages with separate subject and body fields. Accordingly, this report uses the precise term actionable email-content classification. The original labels are retained: Yes denotes Actionable and No denotes Non-Actionable.

2. Problem Statement

Given a text snippet from an email dataset, the system must assign one of two labels: Actionable (Yes) or Non-Actionable (No). The positive class is Actionable (Yes). The experimental question is how the three implemented approaches perform when trained and evaluated using the same cleaned data split and held-out test set.

3. Objectives

- Prepare a cleaned, labeled dataset and create common stratified train, validation, and test splits.
- Implement three models representing sparse statistical features, pretrained word embeddings with sequential modeling, and pretrained Transformer fine-tuning.
- Evaluate every model on the same held-out test split using accuracy, precision, recall, F1-score, ROC-AUC, and confusion-matrix counts.
- Compare the final test results while treating Actionable (Yes) as the positive class.

4. Dataset

The source dataset is the Parakweet Labs Email Intent Dataset file Ask0729-fixed.txt. In the original project dataset, there are 3,657 examples: 1,719 Actionable (Yes) and 1,938 Non-Actionable (No). These figures describe the source data before the project cleaning pipeline is applied.

For the project experiments, the preprocessing code produced 3,651 final examples after removing six records. The retained experimental dataset contains 1,718 Yes examples and 1,933 No examples. Stratified splitting with seed 42 produced 2,555 training examples, 548 validation examples, and 548 test examples. The test split contains 258 Yes and 290 No examples.

Dataset stage	Total	Actionable (Yes)	Non-Actionable (No)
Source dataset	3,657	1,719	1,938
Processed experimental data	3,651	1,718	1,933
Held-out test split	548	258	290

5. Methodology

All three pipelines use the same finalized train, validation, and test files. The training split is used to fit model parameters; the validation split is used for validation and checkpoint selection where implemented; and the test split is reserved for final evaluation. Model outputs are converted to the Actionable class when the predicted probability is at least 0.50.

The evaluation module applies the same positive-class definition and metric calculations to each model. This common evaluation procedure supports a direct comparison of the reported held-out-test results.

6. Preprocessing

The project preprocessing pipeline normalizes text using Unicode NFKC normalization and HTML decoding. It replaces URLs with \$LINK, email addresses with \$EMAIL, and numeric expressions with \$NUM. Remaining HTML tags are removed, and whitespace is normalized.

Records that are empty or meaningless after cleaning are removed. The pipeline also removes identical cleaned text that appears with conflicting labels, because the project does not resolve which label would be correct, and removes exact duplicate cleaned text with the same label. The resulting data are then split stratifiably into 70% training, 15% validation, and 15% test partitions.

7. Model Architectures

7.1 TF-IDF + Logistic Regression

The first pipeline represents cleaned text with a TF-IDF vectorizer and classifies it using Logistic Regression. The vectorizer uses lowercase processing, unigram and bigram features, min_df=2, max_df=0.9, sublinear term-frequency scaling, Unicode accent stripping, and no stop-word list. Logistic Regression uses C=1.0, balanced class weights, the liblinear solver, and a maximum of 2,000 iterations. The TF-IDF vocabulary is fitted only on the training split.

7.2 Pretrained Word2Vec + BiLSTM

The second pipeline constructs a vocabulary from the training text and maps token identifiers to a pretrained Word2Vec embedding matrix. The implementation uses sequences up to 64 tokens, a vocabulary frequency threshold of 2, and a maximum vocabulary size of 20,000. The embedding layer is trainable. A one-layer bidirectional LSTM with hidden dimension 128 processes the embeddings; its forward and backward final states are concatenated and passed through dropout (0.3) and a fully connected two-class output layer.

7.3 Pretrained BERT

The third pipeline fine-tunes bert-base-uncased using AutoTokenizer and AutoModelForSequenceClassification with two output labels. Text is tokenized with truncation and max-length padding to 64 tokens. The implementation retains all BERT parameters as trainable and uses the model's classification output for the binary decision.

8. Training

The TF-IDF plus Logistic Regression model is fitted on the training split. The BiLSTM configuration specifies batches of 32, up to 20 epochs, learning rate 0.001, gradient clipping at 5.0, and patience of 4 validation epochs. Its implementation uses cross-entropy loss and the Adam optimizer, saving the checkpoint with the highest validation F1-score.

The BERT configuration specifies batches of 16, up to 4 epochs, learning rate 2e-5, weight decay 0.01, a linear warmup ratio of 0.1, gradient clipping at 1.0, and patience of 2 validation epochs. It uses cross-entropy loss, AdamW optimization, a linear warmup scheduler, and selects the checkpoint with the highest validation F1-score. In both neural pipelines, the held-out test split is evaluated after training using the selected checkpoint.

9. Evaluation Metrics

Accuracy is the proportion of all test examples classified correctly. Precision measures the proportion of predicted Actionable examples that are actually Actionable. Recall measures the proportion of actual Actionable examples that are detected. The F1-score is the harmonic mean of precision and recall. ROC-AUC summarizes probability-ranking performance across classification thresholds.

The confusion matrix is reported with rows as true labels and columns as predicted labels, in the order No then Yes. In this project, a false negative is an Actionable email-content example predicted as Non-Actionable; this is important because it represents actionable content that could be missed.

10. Results and Comparison

Table 2 presents the final held-out-test results from the repository's comparison artifact. Percentages are displayed to two decimal places. No additional experiments or aggregate statistics are inferred beyond these recorded results.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
TF-IDF + Logistic Regression	77.74%	76.98%	75.19%	76.08%	86.07%
Pretrained Word2Vec + BiLSTM	74.82%	71.74%	76.74%	74.16%	86.54%
Pretrained BERT (bert-base-uncased)	84.85%	82.05%	86.82%	84.37%	93.16%

Model	TP	TN	FP	FN
TF-IDF + Logistic Regression	194	232	58	64
Pretrained Word2Vec + BiLSTM	198	212	78	60
Pretrained BERT (bert-base-uncased)	224	241	49	34

11. Discussion

On the shared held-out test set, BERT records the highest values for accuracy (84.85%), precision (82.05%), recall (86.82%), F1-score (84.37%), and ROC-AUC (93.16%). It also has the fewest false negatives (34) and false positives (49) among the three reported systems. Thus, under this experimental setup, the fine-tuned BERT model provides the strongest recorded overall performance and detects the largest number of Actionable examples (224 of 258).

The TF-IDF plus Logistic Regression model provides a competitive non-neural reference, with 77.74% accuracy and 76.08% F1-score. The Word2Vec plus BiLSTM model has a higher recall (76.74%) and ROC-AUC (86.54%) than the TF-IDF baseline, but its recorded accuracy, precision, and F1-score are lower. These statements are limited to the single shared test split and the configurations recorded in the project repository.

12. Limitations

- The source records are primarily individual email sentences or snippets, not full structured emails with separate subject and body fields; conclusions therefore concern email-content actionability.
- The reported comparison is based on one fixed, stratified split generated with seed 42. The repository does not report repeated runs, cross-validation, or statistical significance testing.
- Results reflect the implemented configurations, preprocessing rules, and a maximum input length of 64 tokens for the neural pipelines; they should not be generalized beyond the evaluated dataset without further testing.
- The project results measure offline classification performance. They do not establish performance in a deployed email workflow or with other email sources.

13. Conclusion

MailSense evaluates three NLP approaches for binary actionable email-content classification using a common cleaned dataset split and a common held-out test set. The final recorded comparison shows that fine-tuned bert-base-uncased achieves the strongest results across all reported primary metrics: 84.85% accuracy, 82.05% precision, 86.82% recall, 84.37% F1-score, and 93.16% ROC-AUC. The TF-IDF plus Logistic Regression and pretrained Word2Vec plus BiLSTM models provide documented alternative baselines within the same experimental framework. The conclusions are restricted to the Parakweet Labs Ask0729-fixed.txt data and the project procedures recorded in the repository.

14. References

- [1] Parakweet Labs. Email Intent Dataset, Ask0729-fixed.txt. Available: <https://raw.githubusercontent.com/ParakweetLabs/EmailIntentDataSet/master/src/resources/Ask0729-fixed.txt>. Accessed for this project dataset.
- [2] MailSense project repository. README.md; data_quality_report.json; final_test_comparison.csv; model configuration files; preprocessing, training, and evaluation source files. Internal project artifacts used as the source of truth for this report.